

From Patterns to Predictions: A Shapelet-Based Framework for Directional Forecasting in Noisy Financial Markets

Juwon Kim
Pohang University
of Science and Technology
Pohang, Republic of Korea
juwona@postech.ac.kr

Hyunwook Lee
Pohang University
of Science and Technology
Pohang, Republic of Korea
gusdnr0916@unist.ac.kr

Hyotaek Jeon
Pohang University
of Science and Technology
Pohang, Republic of Korea
taek98@postech.ac.kr

Seungmin Jin
Ulsan National Institute
of Science and Technology
Ulsan, Republic of Korea
skyjin@unist.ac.kr

Sungahn Ko*
Pohang University
of Science and Technology
Pohang, Republic of Korea
sungahn@postech.ac.kr

Abstract

Directional forecasting in financial markets requires both accuracy and interpretability. Before the advent of deep learning, interpretable approaches based on human-defined patterns were prevalent, but their structural vagueness and scale ambiguity hindered generalization. In contrast, deep learning models can effectively capture complex dynamics, yet often offer limited transparency. To bridge this gap, we propose a two-stage framework that integrates unsupervised pattern extraction with interpretable forecasting. (i) **SIMPC** segments and clusters multivariate time series, extracting recurrent patterns that are invariant to amplitude scaling and temporal distortion, even under varying window sizes. (ii) **JISC-Net** is a shapelet-based classifier that uses the initial part of extracted patterns as input and forecasts subsequent partial sequences for short-term directional movement. Experiments on Bitcoin and three S&P 500 equities demonstrate that our method ranks first or second in **11 out of 12 metric-dataset combinations**, consistently outperforming baselines. Unlike conventional deep learning models that output buy-or-sell signals without interpretable justification, our approach enables transparent decision-making by revealing the underlying pattern structures that drive predictive outcomes.

CCS Concepts

• **Mathematics of computing** → **Time series analysis**; • **Computing methodologies** → *Neural networks*; • **Theory of computation** → *Unsupervised learning and clustering*.

Keywords

Financial Forecasting; Multivariate Time Series; Pattern Extraction; Unsupervised Learning; Explainable ML

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

CIKM '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2040-6/2025/11

<https://doi.org/10.1145/3746252.3761250>

ACM Reference Format:

Juwon Kim, Hyunwook Lee, Hyotaek Jeon, Seungmin Jin, and Sungahn Ko. 2025. From Patterns to Predictions: A Shapelet-Based Framework for Directional Forecasting in Noisy Financial Markets. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*, November 10–14, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3746252.3761250>

1 Introduction

Accurately forecasting market movements such as stock prices with deep learning is difficult due to the frequent volatility and low signal-to-noise ratio inherent in financial markets. For instance, Rossi [23] notes that exchange rate forecasts often fail to outperform a simple random walk. Even advanced models often fail to outperform naive baselines using only the last observed price [30]. These challenges have led to numerous stock prediction approaches, broadly classified into two types: (i) internal time series-based models, which use only historical prices and technical indicators [7, 22]; and (ii) hybrid models, which combine internal price dynamics with external signals such as financial news, social media, or macroeconomic indicators [5, 13]. In this study, following the efficient market hypothesis that asset prices reflect all available information, we focus solely on price data to capture the intrinsic dynamics of stock price movements. Given the universal availability of price data and independence from exogenous inputs, this approach is applicable across diverse stocks and markets. However, price-only modeling faces three major challenges. First, financial time series are highly noisy with weak long-term signals, making it difficult to separate meaningful patterns from random fluctuations. Second, recurrent patterns appear at varying scales and durations. We term these variations **amplitude-scaling**, denoting proportional changes in both magnitude and duration, and **temporal-distortion**, referring to non-uniform changes in the timing or speed of specific segments within a pattern. These two types encompass the canonical distortions explored in prior work [6, 11], offering a structured perspective for interpreting recurrent patterns within financial data. Third, deep neural networks can model complex market dynamics but offer limited **interpretability** due to their black-box nature.

To overcome these limitations, we introduce a two-stage framework that extracts and leverages interpretable patterns from noisy

multivariate financial time series. In the **pattern extraction** stage, we propose the *Selective Invariant Multivariate Pattern Clustering* (SIMPC) module, which segments and clusters multivariate subsequences using *Dynamic Time Warping* (DTW) [2]. SIMPC identifies patterns invariant to amplitude scaling and temporal distortion, even under dynamic window conditions. By incorporating multivariate inputs, it enables fine-grained contextual differentiation. In the **pattern detection** stage, we propose the *Joint-variable Invariant Shapelet-Classification Network* (JISC-Net), which combines DTW with a *Multi-length-input dilated causal Convolutional Neural Network* (Mdc-CNN) encoder [12] to learn representations robust to temporal distortions. JISC-Net identifies and filters meaningful pattern instances through a two-stage filtering process, excluding noisy or non-pattern segments. Empirical evaluations on four financial assets show that our method significantly improves directional accuracy and trading performance over competitive baselines. By focusing on localized subsequences that align with empirically extracted pattern structures, rather than modeling the entire time series, the proposed framework enhances both interpretability and practical utility in real-world financial forecasting.

2 Related Work

Time Series Forecasting Models. Relevant forecasting models include multivariate approaches and stock-specific deep learning methods. **Multivariate time series forecasting** has made substantial progress in modeling complex temporal and cross-variable dependencies. FEDformer [36] extends Transformer-based forecasting by leveraging seasonal-trend decomposition in the frequency domain, while PatchTST [21] divides time series into patch tokens and applies channel-independent self-attention to capture long-term dependencies. In addition, TimesNet [25] employs temporal 2D variation blocks to capture multiscale patterns in the time-frequency space. Despite their effectiveness on standard benchmarks, these models often underperform on noisy financial time series due to their reliance on dense temporal encoding, which increases the risk of overfitting. In response, **stock-specific architectures** have been proposed to address these domain challenges. The *State-Frequency Memory* (SFM) network [33] decomposes temporal data into frequency components to model stock price dynamics but struggles with non-stationary conditions. Stock-Mixer [7] utilizes a compact MLP for patch-based mixing of multivariate inputs, capturing local patterns. MASTER [14] integrates a market-guided gating mechanism within a Transformer framework, dynamically selecting features and aggregating information across stocks to capture market-driven relationships. More recently, SAMBA [19] combines continuous-state sequence modeling with a dynamic stock graph for scalable long-term forecasting. Even with these advancements, similar to time series models, stock-specific approaches remain black boxes with limited transparency, rarely utilizing human-interpretable patterns for predictive decisions.

Pattern Clustering in Time Series. Unsupervised and weakly supervised methods have been developed to identify recurring temporal structures. For example, *Deep Temporal Contrastive Clustering* (DTCC) [34] introduces contrastive learning into time series clustering for the first time, combining instance-level and cluster-level contrastiveness within a self-supervised autoencoder framework.

By jointly optimizing reconstruction, clustering, and contrastive losses, DTCC improves representation quality for general time series clustering tasks. However, it does not specifically address the challenges posed by financial time series, such as amplitude scaling and temporal distortion. A particularly relevant precursor to our approach is the *Scale-Invariant Subsequence Clustering* (SISC) algorithm [9], which partitions financial time series into variable-length segments and clusters them into shape-based groups that are invariant to both amplitude and duration. While effective at identifying latent motifs, SISC is limited to univariate inputs and may admit noisy or irrelevant segments. **Our proposed SIMPC** extends this line of work by accommodating multivariate inputs, initializing clusters with domain-informed prototypes, and explicitly filtering out low-information subsequences.

Shapelet-Based Classification. Shapelet-based methods provide interpretable representations for time series classification by identifying discriminative subsequences. The concept was first introduced by Ye and Keogh [29] for univariate data, and later extended to learnable shapelets [8] and binary representations for multiclass classification [3]. ShapeNet [12] further generalized these methods to multivariate series by incorporating dilated causal convolutions and triplet loss to learn embeddings of variable-length shapelets. Building on this foundation, **our proposed JISC-Net** incorporates a DTW-based triplet loss to enhance robustness against temporal distortions common in financial data, and treats multivariate observations as unified tokens to maintain joint temporal dependencies across variables.

3 Problem Statement

The goal is to identify and detect recurring patterns embedded in noisy **multivariate** time series that exhibit **amplitude-scaling** and **temporal-distortion** characteristics, in a fully **unsupervised** manner, and to predict future directional movement (rise or fall) based on these patterns. This is accomplished through two stages: pattern extraction and classification, as illustrated in Figure 1.

Pattern Extraction. Let $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{T \times D}$ be a multivariate time series, where $\mathbf{x}_t \in \mathbb{R}^D$ denotes the feature vector at time t . During pattern extraction, SIMPC segments $\mathbf{X}$ and derives P pattern clusters, denoted as $C = \{C_1, C_2, \dots, C_P\}$. Each cluster C_p comprises multiple temporally contiguous subsequences associated with pattern p :

$$C_p = \{\mathbf{s}_{p,1}, \mathbf{s}_{p,2}, \dots\}, \quad \mathbf{s}_{p,j} \in \mathbb{R}^{\ell \times D}$$

where $\mathbf{s}_{p,j}$ denotes the j th subsequence in cluster C_p , with variable length $\ell \in [L_{\min}, L_{\max}]$. Cluster sizes may vary depending on the frequency of pattern occurrences in the time series. Each subsequence $\mathbf{s}_{p,j}$ is a contiguous segment of $\mathbf{X}$, starting at a specific time step. To ensure non-overlapping pattern boundaries, we require that the start time of each subsequence be unique across all clusters. Since the P clusters may contain redundant or semantically similar patterns, we perform cluster merging in the final stage of SIMPC, which results in a refined pattern set of size P' .

Pattern Detection. The classifier JISC-Net is trained on the extracted pattern set C to map an input subsequence $\mathbf{s}_{p,j}$ to a probability distribution over P' pattern types. Importantly, only the initial γ portion of each subsequence is used as input to JISC-Net,

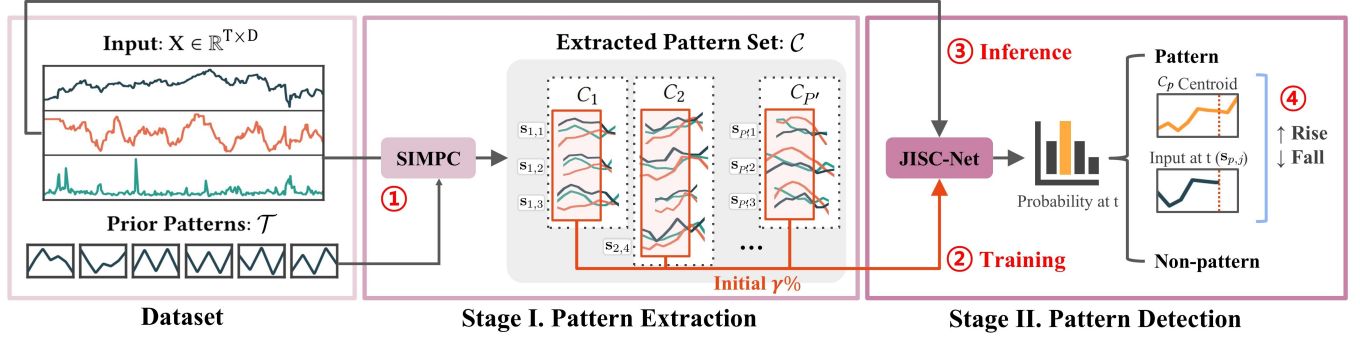


Figure 1: Overview of the proposed two-stage framework. Stage 1: SIMPC extracts robust, recurring patterns from multivariate time series under amplitude scaling and temporal distortion. Stage 2: JISC-Net performs pattern classification using the initial portion of each pattern subsequence and predicts the directional movement of its remaining part.

where $\gamma \in (0, 1)$ denotes the proportion of the sequence that is utilized during classification. The remaining $(1 - \gamma)$ portion can be reasonably inferred once the pattern is detected, as its structure has already been captured during pattern extraction. To ensure reliable classification predictions, we apply two filtering mechanisms. During training, label-level filtering with the *Kolmogorov-Smirnov* (K-S) test excludes statistically insignificant subsequences from the trading candidate set. During inference, confidence-based thresholding (top- $x\%$) removes ambiguous or non-pattern instances. As a result, JISC-Net can reliably identify and reject inputs that do not correspond to meaningful patterns.

4 SIMPC: Pattern Extraction

Selective Invariant Multivariate Pattern Clustering (SIMPC) extracts patterns by segmenting a multivariate time series into variable-length candidates and clustering only the most informative ones. It extends the *Segmentation-Invariant Subsequence Clustering* (SISC) algorithm [9], which is a K-means++-style algorithm that greedily segments univariate series into variable-length windows and clusters them using DTW. However, SISC suffers from key limitations: it is restricted to univariate inputs, initializes the first centroid randomly as in K-means++, and clusters all segments regardless of informativeness. **SIMPC addresses these issues by** (1) generalizing DTW to multivariate inputs to capture inter-variable dependencies. This enables the model to distinguish semantically distinct behaviors that may appear identical in a single variable, enhancing interpretability through richer multivariate context; (2) incorporating domain knowledge via canonical chart pattern prototypes for centroid initialization (see $\mathcal{T}$ in Fig. 1); and (3) pruning low-information segments and discarding sparse clusters to suppress noise. We proceed by detailing each component of the algorithm.

1. Pre-processing by Kernel Regression. To suppress short-term volatility, we smooth the multivariate input X using Nadaraya-Watson kernel regression [20] for each variable independently.

$$\tilde{x}_t = \frac{\sum_{s=1}^T K_h(t-s) \mathbf{x}_s}{\sum_{s=1}^T K_h(t-s)}, \quad K_h(u) = K\left(\frac{u}{h}\right), \quad (1)$$

where $K(\cdot)$ is a Gaussian kernel and h is the bandwidth. This yields the denoised sequence $\tilde{X}$, subsequently used for clustering.

2. Domain Adaptation via Canonical Chart Patterns. Empirical evidence shows that common behavioral biases recur in similar forms across asset classes, supporting the use of canonical stock rules as a cross-market prior [17]. Motivated by this, we incorporate domain knowledge into clustering by constructing the initial centroid set using *traditional chart patterns* (see Appendix A). This step is performed independently of the main clustering pipeline using a separate historical dataset, from which we extract multivariate time series segments that satisfy canonical chart rules [18]. Prior to extraction, each variable is smoothed with kernel regression (Eq. 1) to mitigate short-term volatility.

We first identify pattern-conforming intervals based on *price* alone, following structural constraints [17], and then collect additional market variables over the same spans. Each extracted segment is independently normalized per variable using Min-Max scaling, since patterns depend on relative shape rather than absolute magnitude. After normalization, all segments are linearly interpolated to a common reference length L . To compute a representative prototype for each chart pattern class, we collect all normalized *price* sequences assigned to that class and aggregate them using *DTW Barycenter Averaging* (DBA). This yields an average price trajectory that preserves structural features while temporally aligning the sequences. The alignment for each sequence is captured in its own warping matrix, which records how that price sequence is matched to the class-specific average. We then multiply its companion variables by each sequence’s warping matrix, thereby temporally synchronizing all dimensions of the multivariate sequence with the average price trajectory. Finally, for each pattern, we compute the multivariate prototype $\mathcal{T}_i$ by taking the element-wise mean across all aligned multivariate segments.

3. Adapted K-means++ Initialization. A user-defined number m of prototypes $\{\mathcal{T}_1, \dots, \mathcal{T}_m\}$ is retained as **fixed seeds**. Since SIMPC is designed to form a total of P pattern clusters, the remaining $P - m$ centroids are initialized using this procedure. We enumerate all valid start indices $s \in \{1, \dots, T - L_{\max} + 1\}$ in the smoothed input $\tilde{X}$ to extract candidate segments of length $L_{\max}$:

$$\mathbf{q}_{s, L_{\max}} = \tilde{X}_{s:s+L_{\max}} \in \mathbb{R}^{L_{\max} \times D},$$

Each segment $\mathbf{q}_{s, L_{\max}}$ is Min-Max normalized per variable to yield $\hat{\mathbf{q}}_{s, L_{\max}}$, emphasizing relative shape changes over absolute values.

We then compute the minimum DTW distance between each normalized candidate $\hat{\mathbf{q}}_{s,L_{\max}}$ and the current centroid set $\mathcal{P}$:

$$d_s = \min_{C \in \mathcal{P}} d_{\text{DTW}}(\hat{\mathbf{q}}_{s,L_{\max}}, C).$$

Sampling probabilities are defined as $p_s = d_s / \sum_{i \in \mathcal{S}} d_i$, where $\mathcal{S}$ denotes the set of candidate start indices not yet selected as centroids. One new centroid is drawn according to this distribution, added to $\mathcal{P}$, and the distances d_s and probabilities p_s are updated accordingly. The procedure repeats until $|\mathcal{P}| = P$.

4. Iterative Cluster Generation and Centroid Update. With P centroids initialized, SIMPC performs I clustering iterations, each involving greedy segmentation and centroid updates. At each time step t , we consider all segment lengths $\ell \in [L_{\min}, L_{\max}]$ and extract candidate segments $\mathbf{q}_{t,\ell} = \tilde{\mathbf{X}}_{t:t+\ell} \in \mathbb{R}^{\ell \times D}$. After per-variable Min-Max normalization, each segment $\hat{\mathbf{q}}_{t,\ell}$ is compared against all centroids, and the one with the minimum DTW distance is selected:

$$(d^*, C^*, \ell^*) = \arg \min_{C \in \mathcal{P}, \ell} d_{\text{DTW}}(\hat{\mathbf{q}}_{t,\ell}, C).$$

If the minimum distance d^* is less than or equal to the DTW threshold δ , the segment is assigned to C^* . The time index is then advanced by a fixed stride length. After one full pass over $\tilde{\mathbf{X}}$, clusters with at least κ members are recompressed using DBA, while smaller clusters are discarded. The resulting barycenters are then renormalized via per-variable Min-Max scaling and retained as updated centroids. If the number of clusters falls below P , the K-means++ initialization (step 3) is repeated to maintain $|\mathcal{P}| = P$, using the updated centroids as fixed seeds. During this step, start indices s that have already been assigned to existing clusters are excluded from consideration. Finally, upon completing I clustering iterations, any pair of centroids (C_i, C_j) such that $d_{\text{DTW}}(C_i, C_j) \leq \delta$ is merged via DBA, yielding the final centroid set $C = \{C_1, \dots, C_{P'}\}$ with $P' \leq P$.

5. Abridged Pseudocode. Algorithm 1 is the SIMPC pseudocode. It outputs a pattern set C , which serves as explicit labels for the subsequent shapelet-based classifier, thereby converting JISC-Net’s pattern detection into a fully supervised learning task.

5 JISC-Net: Pattern Detection

Joint-variable Invariant Shapelet-Classification Network (JISC-Net) extends the *ShapeNet* [12], which is designed to discover shapelets—short, discriminative subsequences that characterize class-specific patterns in multivariate time series. We adopt this shapelet-based approach due to its inherent interpretability, as it enables identification of motifs that directly contribute to classification decisions, a desirable property in financial applications where model transparency is critical. ShapeNet operates by sliding variable-length windows across all input channels, embedding the resulting candidates using a causal CNN, and selecting the most class-informative shapelets through end-to-end training. Among existing shapelet-based models, ShapeNet is particularly well-suited to our task, as its ability to learn variable-length shapelets allows it to flexibly adapt to temporal variations and effectively handle the amplitude-scaling effects commonly observed in financial time series. **JISC-Net differs from the original ShapeNet in two key aspects:** (1) We regard the entire D -dimensional observation $\mathbf{x}_t$ as a *single token*,

Algorithm 1 SIMPC

Require: Denoised input $\tilde{\mathbf{X}} \in \mathbb{R}^{T \times D}$; desired cluster count P ; fixed seeds $\{\mathcal{T}_1, \dots, \mathcal{T}_m\}$; segment-length bounds $(L_{\min}, L_{\max})$; DTW threshold δ ; minimum cluster size κ ; number of iterations I ; stride Δ

```

// —2. Domain Adaptation—
1:  $\mathcal{P}_0 \leftarrow \{\mathcal{T}_1, \dots, \mathcal{T}_m\}$ 
// —3. Adapted K-means++ Initialization—
2:  $\mathcal{P} \leftarrow \mathcal{P}_0$  ▷  $\mathcal{P}$ : Current centroid set
3: while  $|\mathcal{P}| < P$  do
4:   for all unselected start indices  $s$  do
5:      $\hat{\mathbf{q}}_{s,L_{\max}} \leftarrow \text{MinMax}(\tilde{\mathbf{X}}_{s:s+L_{\max}})$ 
6:      $d_s \leftarrow \min_{C \in \mathcal{P}} d_{\text{DTW}}(\hat{\mathbf{q}}_{s,L_{\max}}, C)$ 
7:   end for
8:   Sample  $s^*$  with probability  $p_s = d_s / \sum_{i \in \mathcal{S}} d_i$ 
9:    $\mathcal{P} \leftarrow \mathcal{P} \cup \{\hat{\mathbf{q}}_{s^*,L_{\max}}\}$ 
10: end while
// —4. Iterative Clustering and Centroid Update—
11: for  $\text{iter} = 1$  to  $I$  do
12:    $t \leftarrow 1$ ; initialize empty cluster map  $C$ 
13:   while  $t + L_{\min} \leq T$  do
14:     for all  $\ell \in [L_{\min}, L_{\max}]$  do
15:        $\hat{\mathbf{q}}_{t,\ell} \leftarrow \text{MinMax}(\tilde{\mathbf{X}}_{t:t+\ell})$ 
16:       Compute  $d(C, \ell) \leftarrow d_{\text{DTW}}(\hat{\mathbf{q}}_{t,\ell}, C)$  for all  $C \in \mathcal{P}$ 
17:     end for
18:      $(d^*, C^*, \ell^*) \leftarrow \arg \min_{C, \ell} d(C, \ell)$ 
19:     if  $d^* \leq \delta$  then  $C[C^*] \cup = \tilde{\mathbf{X}}_{t:t+\ell^*}$ 
20:     end if
21:      $t \leftarrow t + \Delta$ 
22:   end while
23:   for all  $C \in \mathcal{C}$  do
24:     if  $|C[C]| \geq \kappa$  then  $C \leftarrow \text{MinMax}(\text{DBA}_{\text{DTW}}(C[C]))$ 
25:     else remove  $C$  from  $\mathcal{P}$ 
26:   end if
27: end for
28:   Repeat lines 3–10 until  $|\mathcal{P}| = P$ 
29: end for
30: Merge centroids with  $d_{\text{DTW}} \leq \delta$ 
31: return Final centroid set  $C$ 

```

rather than processing each channel separately. This preserves cross-variable interactions that are critical in multivariate time series. (2) We replace the Euclidean distance used for triplet loss function in ShapeNet with a DTW-based distance, endowing the encoder with robustness to temporal distortion. These changes allow JISC-Net to capture richer inter-variable dynamics while remaining resilient to heterogeneous temporal scales.

1. Mdc-CNN Encoder with DTW-based Triplet Loss. We consider all pattern-labeled subsequences $\mathbf{s}_{p,j} \in C_p$ from clusters $C = \{C_1, \dots, C_{P'}\}$, output by SIMPC. In pattern detection, only the initial γ fraction of each subsequence serves as input. Figure 2 shows the encoder training process, and Figure 3 its architecture. Each sequence is linearly interpolated to a fixed length $L (> \gamma\ell)$ and column-wise Min-Max normalized, yielding $\hat{\mathbf{s}}_{p,j} \in \mathbb{R}^{L \times D}$. In order to apply the triplet loss function during the encoder training, we need to specify anchor, positive, and negative points. To achieve this, we iterate over the multiscale slide-ratio set α , where the elements of α lie between 0 and 1. For each $\alpha \in \alpha$, a transposed subsequence $\hat{\mathbf{s}}_{p,j}^\top$ is repeatedly sliced into segments of length αL by

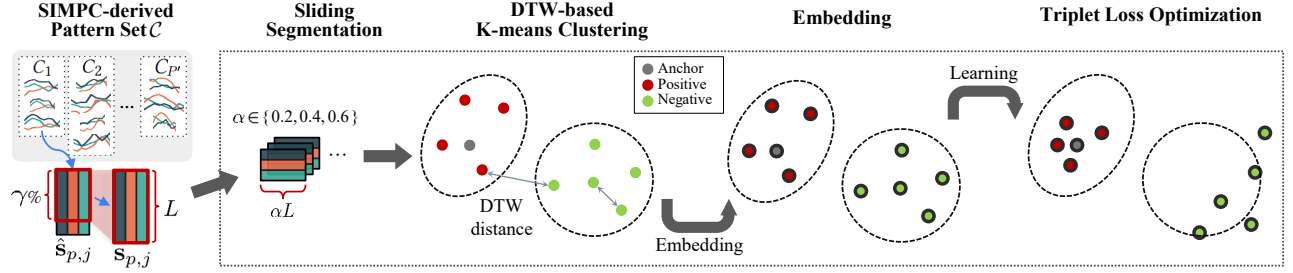


Figure 2: The encoder learning process of JISC-Net

sliding a window with a fixed stride. We then cluster the segments for each α using the *dependent DTW-based K-means* ($K=2$). This multiscale slicing increases training instances and exposes the encoder to local patterns at multiple resolutions. This process applies segmentation and clustering to each α and $\hat{s}_{p,j}^\top$ pair individually, ensuring the encoder receives input shaped as $(-1, D, \alpha L)$.

Let c be one of the two clusters produced by DTW-based K-means. For each of the two clusters, we select (i) the segment that is located most centrally within the cluster as the *anchor* A , (ii) the $\lceil |c|/5 \rceil$ segments closest to the anchor as the *positive set* S^+ , and (iii) the $\lceil |c|/5 \rceil$ segments farthest from the anchor, drawn from the opposite cluster $\bar{c}$, as the *negative set* S^- . These selected anchor, positive, and negative points are then embedded through the encoder, and during the optimization process using the triplet loss function, the encoder learns meaningful feature representations.

The Mdc-CNN encoder f_θ maps each input segment of shape $(D, \alpha L)$ to an embedding $z \in \mathbb{R}^h$, where $z_a, z_p, z_n = f_\theta(A), f_\theta(S^+), f_\theta(S^-)$ with masking applied to accommodate the varying sizes of S^+ and S^- . We then compute the distances d_{ap} (anchor-positive), d_{an} (anchor-negative), and intra-class distances d_{intra}^+ and d_{intra}^- .

$$d_{ap} = \frac{1}{|S^+|} \sum_j \|z_a - z_{p_j}\|_2, \quad d_{an} = \frac{1}{|S^-|} \sum_j \|z_a - z_{n_j}\|_2,$$

$$d_{intra}^+ = \frac{1}{|S^+|^2} \sum_{i,j} \|z_{p_i} - z_{p_j}\|_2, \quad d_{intra}^- = \frac{1}{|S^-|^2} \sum_{i,j} \|z_{n_i} - z_{n_j}\|_2.$$

For a given anchor, the loss is computed as:

$$\ell = \log \left(\frac{d_{ap} + d_{intra}^+ + d_{intra}^-}{d_{an} + \varepsilon} + m \right),$$

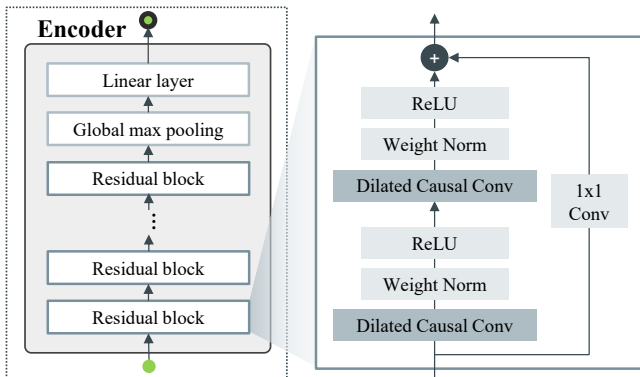


Figure 3: Architecture of the encoder Mdc-CNN

where $\varepsilon = 10^{-6}$ prevents numerical instability and $m = 0.2$ implements a soft margin. This formulation penalizes large anchor-positive distances and weak intra-cluster cohesion while encouraging anchor-negative separation. The total loss is averaged across all segments sampled from $\hat{s}_{p,j}$. By replacing Euclidean distance with DTW, the encoder learns invariance to temporal distortions. Moreover, all variable channels are encoded jointly, preserving cross-variable dependencies.

2. Shapelet Discovery. This phase is dedicated to the discovery of discriminative shapelets using the encoder trained in the preceding phase. The normalized subsequence $\hat{s}_{p,j}^\top$ is segmented at each slide ratio α , in accordance with the procedure described earlier. Each segment is encoded into a fixed-length vector using the trained encoder f_θ . All embeddings corresponding to segments derived from every $(\hat{s}_{p,j}^\top, \alpha)$ pair are subsequently collected into a latent cloud. Since temporal distortion invariance has already been learned by f_θ , we apply basic Euclidean K-means (hyperparameter $K = g$) to the latent cloud, yielding g clusters. For each cluster, we select the embedded segment with the smallest Euclidean distance to the cluster center as the shapelet candidate. The corresponding pre-embedding segment is then traced to recover the raw multivariate curve, denoted as $Shp_c \in \mathbb{R}^{D \times \alpha L}$, where $c \in \{1, \dots, g\}$ indexes the shapelet clusters.

To ensure that each of the g clusters represents a class-consistent and meaningful group, we evaluate its **purity**, which is computed using the embedded segments. The purity of a cluster is determined by the proportion of segments within the cluster that share the same label. The label refers to the pattern to which the segment belongs, as identified by the output of SIMPC. The purity is calculated as the ratio of the number of segments with the most frequent label in the cluster to the total number of segments in that cluster. A cluster is retained if its purity exceeds a threshold of $\frac{1}{P'}$, where P' is the total number of patterns. To rank the retained clusters, we define a **utility score** U_c that quantifies the distinctiveness and representativeness of each shapelet candidate. This score is used solely for ranking and does not affect prediction. For shapelet cluster c , the utility score is computed as

$$U_c = |C_c| \cdot \sum_{c' \neq c} \|Shp_c - Shp_{c'}\|_2^2,$$

where $|C_c|$ is the number of segments in cluster c . This formulation favors candidates from larger clusters that are well-separated from others in latent space.

3. SVM classifier. In this phase, shapelet candidates are used to train a *Support Vector Machine* (SVM) for pattern classification. For each shapelet candidate $\text{Shp}_c \in \mathbb{R}^{D \times L_c}$, where L_c denotes the length of the shapelet, we compute the minimum DTW distance between Shp_c and all segments of the same length in $\hat{s}_{p,j}^\top$. Concretely, we slide a window of length L_c over $\hat{s}_{p,j}^\top$, extract each segment $\hat{s}_{p,j}^\top[t : t + L_c]$, and compute the DTW distance between this segment and the raw shapelet Shp_c . The final feature vector is

$$\Phi\left(\hat{s}_{p,j}^\top\right) = \left[\min_t \text{DTW}\left(\hat{s}_{p,j}^\top[t : t + L_c], \text{Shp}_c\right)\right]_{c=1}^g \in \mathbb{R}^g,$$

where g denotes the number of retained shapelets. The feature vectors from all subsequences $\hat{s}_{p,j}^\top$, along with their corresponding pattern labels provided by SIMPC, are then used to train a linear SVM. The trained classifier outputs a probability distribution $P \in [0, 1]^{P'}$ over the P' pattern types. Each subsequence is classified into the pattern with the highest probability, denoted as $p_{\max}$.

To improve prediction accuracy, we apply filtering during both training and inference. In the **training stage**, we use the Kolmogorov-Smirnov (K-S) test to identify non-discriminative pattern labels. For each label, we collect $p_{\max}$ values from correctly and incorrectly classified subsequences, compute cumulative distributions over discretized intervals, and apply the K-S test. Labels with p-values above 0.05 are deemed non-discriminative and treated as non-patterns. Filtering is performed using only training data to prevent information leakage. In the **inference stage**, we retain only the top $x\%$ of test subsequences sorted by $p_{\max}$, filtering out low-confidence predictions as non-patterns.

Note that only the initial $\gamma\ell$ portion of each pattern-classified subsequence $\hat{s}_{p,j}^\top$ serves as input to the classifier, where γ is a tunable hyperparameter. The detection of a pattern match signifies both pattern identification and enables a directional forecast for the remaining $(1 - \gamma)\ell$ portion.

6 Experiments

We design our experiments to address four research questions (RQs) on the roles of multivariate input, baseline comparisons, confidence-based filtering, and interpretability within the proposed two-stage framework. Code for our framework is available on GitHub¹.

6.1 Experimental Setup

Data. We evaluate the framework on two financial domains: cryptocurrency (BTC/USD) and equities (S&P 500). The equities dataset comprises three high-cap stocks from different S&P 500 sectors: AAPL (Technology), BRK.B (Financials), and XOM (Energy). Each asset is represented by closing price, trading volume, and relative strength index (RSI). We split the datasets into training (BTC/USD: 2014/01/01–2021/12/31; stocks: 2008/01/01–2018/12/31), validation (BTC/USD: 2022/01/01–2023/01/31; stocks: 2019/01/01–2021/07/30), and testing (BTC/USD: 2024/02/01–2025/05/13; stocks: 2021/08/01–2025/05/13) periods. All models, including baselines, are trained and evaluated using the same input features to ensure fair comparison.

Table 1: Clustering outcomes across input configurations.

Input	# Subseq.	Avg. Dist.	Min. Dist.	Below δ
C	658	1.6428	0.2174	(3,4), (3,5), (1,7)
CV	657	1.698	0.759	None
CVR	642	1.743	0.898	None

Hyperparameter. We set the pattern length to approximately 20 days with $L_{\min} = 18$ and $L_{\max} = 22$. SIMPC extracts $P = 8$ patterns, seeded with $m = 6$ canonical chart prototypes, using a DTW threshold $\delta = 2.3$ and minimum cluster size $\kappa = 40$. JISC-Net uses the first $\gamma = 0.8$ portion of each subsequence for classification, enabling inference over the remaining about 4 steps. Each subsequence is linearly interpolated to length $L = 100$. Multiscale shapelets are generated with sliding ratios $\alpha = \{0.2, 0.4, 0.6\}$, and the number of shapelets is set to $g = 10$.

RQ1: Effect of Input Variables

We investigate how different combinations of input variables affect the structure and separability of patterns extracted by the SIMPC algorithm. Our comparison includes: (i) closing price only (C), (ii) closing price and volume (CV), and (iii) closing price, volume, and RSI (CVR). Since dependent DTW accumulates Euclidean distances across dimensions, we scale the clustering threshold δ according to input dimensionality: $\delta = 2.3/\sqrt{3}$ for 1D inputs, $\delta = 2.3/\sqrt{2}$ for 2D, and $\delta = 2.3$ for 3D. Table 1 summarizes the number of subsequences extracted as candidate patterns and the pairwise DTW distances between centroids, before applying the final merging step based on δ . To ensure a fair comparison across input dimensionalities, both the average and minimum centroid-level DTW distances (Avg. Dist., Min. Dist.) are normalized by $\sqrt{d}$. The number of extracted subsequences is comparable across all configurations (C: 658, CV: 657, CVR: 642), with an average of approximately 80 subsequences per cluster. However, clustering quality differs significantly depending on input dimensionality. A larger average DTW distance indicates greater diversity among pattern centroids, and a larger minimum distance implies better separation even between the most similar clusters. In the C-only case, several centroid pairs (e.g., (3, 4), (3, 5), (1, 7)) show distances below the threshold δ , suggesting potential structural overlap. These results confirm that adding volume and RSI as input variables enhances the distinctiveness of extracted pattern centroids. In particular, the CVR configuration yields the most diverse and well-separated centroids, even before any merging step, highlighting the importance of multivariate inputs in capturing structural variation in financial time series.

RQ2: Baseline Comparison

Evaluation Metrics. We report four metrics [15]: F1-score (F1), Win-Loss Ratio (WLR), Average Return (AR), and Total Return with fees (TRwf). F1 captures classification performance under class imbalance. WLR measures the ratio of correctly predicted directional trades to incorrect ones, while AR quantifies the expected profitability per trade. TRwf indicates cumulative returns after transaction costs. All metrics except TRwf ignore fees.

¹<https://github.com/kimjuwon-coder/SIMPC-JISCNet>

Table 2: Evaluation metrics for classification and trading performance. Bold and underline indicate the best and second-best results among baselines and Ours-T@100. The red box highlights the overall best performance in WLR, AR, and TRwf.

Methods	BTC/USD					AAPL				
	F1 [↑]	WLR [↑]	AR [↑]	TRwf [↑]	#Trades	F1 [↑]	WLR [↑]	AR [↑]	TRwf [↑]	#Trades
Random	0.478	0.937	-0.256	-0.842	185	0.519	1.073	-0.016	-0.249	115
LightGBM	0.392	1.229	0.823	1.152	185	0.368	1.130	0.345	0.167	115
DoubleEnsemble	0.267	0.779	-0.819	-1.885		0.309	0.917	-0.090	-0.334	
ADARNN	0.398	<u>1.256</u>	<u>0.966</u>	<u>1.417</u>		0.358	1.091	0.078	-0.141	
ALSTM	0.493	1.056	0.590	0.721		0.358	1.091	0.320	0.138	
TCTS	0.386	1.202	0.920	1.332		0.348	1.054	0.065	-0.156	
Tabnet	0.398	<u>1.256</u>	0.897	1.289		0.338	1.018	-0.129	-0.379	
Transformer	0.487	0.983	-0.113	-0.359		0.487	0.983	-0.113	-0.359	
DLinear	0.511	1.102	0.399	0.369	185	<u>0.513</u>	1.396	<u>0.499</u>	<u>0.343</u>	115
Autoformer	0.490	1.091	-0.017	-0.249		0.490	1.091	-0.017	-0.249	
FEDformer	0.497	1.229	0.217	0.031		0.401	0.983	0.380	0.207	
Informer	<u>0.532</u>	1.056	0.523	0.598		0.458	1.255	0.211	0.013	
iTransformer	0.456	1.079	-0.215	-0.768		0.460	1.054	0.292	0.106	
PatchTST	0.498	<u>1.256</u>	0.522	0.597		0.453	1.396	0.472	0.312	
TimesNet	0.466	1.033	0.322	0.226		0.464	<u>1.500</u>	0.420	0.253	
Ours-T@100	0.594	1.534	1.066	1.602	185	0.541	1.614	0.602	0.462	115
Ours-T@80	0.591	<u>1.561</u>	1.261	1.549	146	0.506	1.583	0.557	0.332	93
Ours-T@60	0.567	1.458	1.265	1.257	118	0.543	2.000	0.688	0.381	78
Ours-T@40	0.511	1.452	<u>1.576</u>	1.046	76	0.565	2.375	0.747	0.295	54
Ours-T@20	0.407	1.375	1.196	0.378	38	0.550	<u>2.429</u>	<u>1.166</u>	0.232	24

Methods	BRK.B					XOM				
	F1 [↑]	WLR [↑]	AR [↑]	TRwf [↑]	#Trades	F1 [↑]	WLR [↑]	AR [↑]	TRwf [↑]	#Trades
Random	0.544	1.146	-0.043	-0.539	222	0.523	1.075	0.094	-0.235	221
LightGBM	0.434	1.220	0.269	0.153	222	0.499	1.210	0.429	0.506	221
DoubleEnsemble	0.261	0.748	-0.282	-1.070		0.282	0.811	-0.288	-1.079	
ADARNN	0.411	1.312	0.308	0.240		0.403	1.278	0.324	0.275	
ALSTM	0.406	1.289	0.295	0.210		0.393	1.232	0.300	0.221	
TCTS	0.422	1.362	0.297	0.215		0.425	1.376	0.405	0.454	
Tabnet	0.443	1.337	<u>0.344</u>	0.320		0.502	1.255	0.323	0.271	
Transformer	0.492	1.413	0.256	0.125		0.495	1.429	0.756	1.228	
DLinear	<u>0.597</u>	1.056	-0.002	-0.448	222	0.537	1.376	0.590	0.861	221
Autoformer	0.581	1.220	0.088	-0.248		0.556	0.973	0.153	-0.104	
FEDformer	0.591	1.440	0.217	0.037		0.530	1.085	0.153	-0.103	
Informer	0.568	1.289	0.218	0.041		0.545	1.376	0.535	0.740	
iTransformer	0.566	1.523	0.253	0.117		0.521	1.232	0.467	0.590	
PatchTST	0.601	1.155	0.064	-0.302		0.540	1.146	-0.024	-0.494	
TimesNet	0.550	1.312	0.296	0.213		0.534	1.255	0.448	0.547	
Ours-T@100	0.573	<u>1.467</u>	0.608	0.907	222	0.573	<u>1.402</u>	0.593	0.868	221
Ours-T@80	0.561	1.392	0.543	0.608	177	0.606	1.603	0.889	<u>1.219</u>	177
Ours-T@60	0.550	1.315	0.647	0.559	125	0.625	1.698	1.030	1.187	143
Ours-T@40	0.544	1.289	0.556	0.310	87	0.624	1.667	1.005	0.708	88
Ours-T@20	0.545	1.316	<u>0.681</u>	0.212	44	0.723	<u>2.625</u>	<u>1.567</u>	0.396	29

Trading Protocol. All experiments follow a representative trading protocol [31, 32] designed to reflect realistic market conditions. Every 4 days during the test period, a position—long or short—is opened based on the predicted 4-day movement and closed at the end of that window. Each trade is executed with a fixed amount, and a 0.1% transaction fee is applied at both entry and exit. To ensure fair comparison, all models are restricted to the same number of trades as our method, preventing frequency-related distortions in return-based metrics. For all baselines except Random, trades are executed only when the prediction confidence is among the top k at fixed 4-day intervals, where k equals the number of trades in our method. For the Random strategy, we randomly select k trading points from the same 4-day intervals to match the number of trades.

This ensures that all models operate under equivalent market exposure and decision frequency, while accounting for prediction confidence.

Baselines. We compare our framework against three baseline categories[31]: (1) **Random**, (2) **Quantitative Investment Methods** frequently employed in Microsoft’s open-source quantitative research platform Qlib [28]: LightGBM[10], DoubleEnsemble[32], ADARNN[4], ALSTM[22], TCTS[27], TabNet[1], Transformer[24], and (3) a recent **Time-series Methods**: DLinear[30], Autoformer[26], FEDformer[36], Informer[35], iTransformer[16], PatchTST[21], and TimesNet[25].

Performance Analysis. Table 2 summarizes the binary classification and trading outcomes on four assets. We denote Ours-T@x as our method with top-x% confidence filtering during inference. Without confidence filtering (Ours-T@100), our method achieves the **best performance** in 8 out of 12 metric–asset combinations and ranks second-best in 3 of the remaining 4, considering the primary metrics (F1, WLR, AR) while excluding TRwf due to its redundancy with AR. Regarding profitability (AR), Ours-T@100 records the highest average return on three assets and the second highest on the fourth. The fee-adjusted total return (TRwf) also remains positive and maximal, demonstrating practical economic value even under conservative cost assumptions. Our model and the baselines execute the same number of trades, so the performance improvement does not arise from reducing trade frequency or selective execution, but from genuinely superior signal quality. These advantages come from the architectural properties of our model. The proposed framework postpones prediction until a recurring pattern with temporal distortion is reliably detected, allowing the model to bypass high-noise segments that typically mislead baseline models. By focusing on a limited, explainable subset of time steps with high predictive confidence, rather than continuously issuing low-confidence signals, the framework demonstrates statistically and economically significant improvements in noisy financial environments.

RQ3: Two-Stage Filtering in JISC-Net

We analyze the two-stage filtering process in JISC-Net that operates on the output of the SVM-based pattern classifier. During training, it filters pattern labels with low statistical separability via the Kolmogorov-Smirnov (K-S) test, and during inference, it imposes a dynamic threshold to retain only high-confidence predictions.

Stage 1: Per-label filtering based on the K-S test. We first apply the K-S test to each pattern label to evaluate whether prediction confidence can effectively distinguish correct from incorrect classifications. A label is retained only if the p-value of the K-S test is below 0.05. Table 3 presents the p-values for XOM labels, revealing that label 7 is the only one that failed the test and was reclassified as a non-pattern. Applying the K-S filter removes 13.90% of test subsequences in BTC/USD, 41.47% in AAPL, 8.74% in BRK.B, and 5.60% in XOM, corresponding to subsequences associated with pattern labels that exhibit low separability.

Table 3: K-S test p-values per label (XOM dataset).

Label	0	1	2	3	4	5	6	7
p-value	1.98e-2	1.00e-4	5.00e-4	2.16e-10	1.32e-6	1.76e-2	1.93e-3	7.05e-1

Stage 2: Global thresholding based on confidence. In the second stage, we sort all predictions by their maximum class probability and retain only the top x% for final inference. Table 2 compares Ours-T@100, T@80, T@60, and T@20 using win-loss ratio (WLR), average return (AR), and fee-adjusted total return (TRwf). We observe that moderately aggressive thresholds provide a favorable balance between return and risk. For instance, on XOM, T@80 achieves a strong AR (0.889) and the highest TRwf (1.219) across all variants, while maintaining a reasonable number of trades

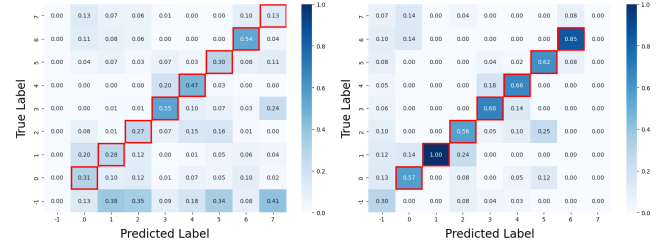


Figure 4: Column-normalized confusion matrices for XOM before (Left) and after (Right, T@30) two-stage filtering. Red boxes mark correct predictions; label -1 indicates non-patterns assignments.

(177). This indicates that filtering out low-confidence predictions can improve profitability without significantly reducing coverage. In contrast, more aggressive filtering (e.g., T@20 on XOM) yields the highest AR (1.567) but significantly reduces profitability (0.396) due to sparse trades (29).

Qualitative impact on confusion structure. Figure 4 illustrates the effect of the two-stage filtering pipeline on classification consistency. The confusion matrix is normalized column-wise to reflect the distribution of true class assignments for each predicted label. Without any filtering, the model frequently misclassifies instances across classes, including assigning noise (label -1) to pattern classes and vice versa. After applying the K-S filtering, label 7—whose p-value exceeded 0.05 in the K-S test (Table 3)—is treated entirely as noise, reducing some misclassifications. Following the top x% thresholding (T@30), the confusion matrix becomes more distinctly diagonal, indicating that the majority of noisy sequences are confidently rejected and pattern ambiguity is further minimized. Overall, the two-stage filtering substantially improves both prediction precision and reliability in pattern classification.

RQ4: Pattern-Based Prediction Analysis

To interpret the model’s decision-making process, we conduct a case study on the XOM asset by analyzing several instances of both successful and failed trades executed by JISC-Net. For each case, we examine: (i) the input sequence used for classification (the initial 16 time steps), (ii) the corresponding pattern predicted by JISC-Net, and (iii) the subsequent 4-step actual price trajectory following the trade. Figure 5 presents **three success cases** in which the model correctly predicted the directional movement, resulting in profitable trades. In all examples, we observe strong alignment between the input sequence and the predicted pattern across all input variables. For instance, in the first case (predicted as pattern 3), the input sequence exhibits a steep decline in price, a surge in trading volume, and a gradually falling RSI. The predicted pattern mirrors these characteristics and projects a further mild decrease in price, sustained high volume, and persistently low RSI in the following 4 steps. The actual market trajectory closely follows this projection, indicating that the model successfully anticipated the short-term price behavior. In contrast, Figure 6 shows **three failure cases** where the model’s prediction led to losses. In the first two cases (predicted as pattern 0 and pattern 2), although the initial portion of the input sequence aligns moderately well with the predicted

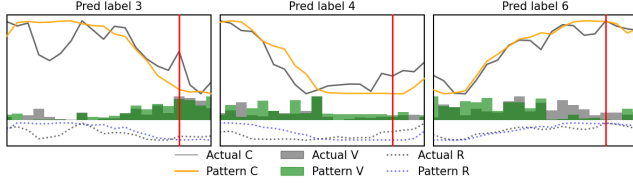


Figure 5: Successful prediction cases. The vertical red line marks the 16th ($\gamma = 0.8$) day, when trading is triggered.

pattern, the subsequent market movement deviates significantly from the anticipated trajectory, especially within the 4-day trading window. This suggests that the model may have overfit to the early segment of the input and failed to account for the variability that arises beyond the classification window. One possible remedy is to incorporate probabilistic forecasts of future trajectories rather than relying solely on deterministic directional predictions. Probabilistic modeling can better capture the uncertainty in future price evolution, enabling the model to express multiple plausible outcomes and reduce overconfidence in a single path, thereby mitigating overfitting to specific historical shapes. Exploring this direction could form the basis of future research. The third failure case demonstrates a clear mismatch even at the initial stage. Although the model assigns this instance to pattern 3, the input sequence diverges substantially from the structure of that pattern. This misclassification likely stems from the limitations of Dynamic Time Warping (DTW), which can occasionally yield artificially low distances when aligning sequences that are temporally shifted but structurally dissimilar. To address this, DTW computations could be regularized by incorporating shape-aware penalties such as constraints on local slope or segment-wise curvature, and by introducing local feature consistency checks that penalize misalignment in critical features (e.g., reversal points, peaks, and troughs). These enhancements would encourage more semantically meaningful alignments and reduce spurious pattern matches.

Overall, these visualizations underscore the interpretability of the proposed framework. Unlike black-box models, JISC-Net combined with SIMPC offers explicit insight into the pattern structure that underlies each trading decision. This not only enhances user understanding but also supports human-in-the-loop decision-making, by allowing users to verify the plausibility of a pattern match before executing a trade.

7 Conclusion

We propose a two-stage framework for pattern-based directional prediction in financial time series. The method integrates SIMPC, a DTW-based unsupervised clustering approach that discovers recurring multivariate patterns with dynamic window sizes under amplitude scaling and temporal distortion, with JISC-Net, a shapelet-based classifier that makes interpretable predictions using partial input sequences. Experiments across four asset classes demonstrate that our method consistently outperforms both quantitative investment models and recent deep learning approaches in directional accuracy and trading performance, while also improving interpretability to support human-in-the-loop decision-making. As future work, we aim to explore probabilistic forecasting and shape-aware alignment to further enhance robustness and generalization.

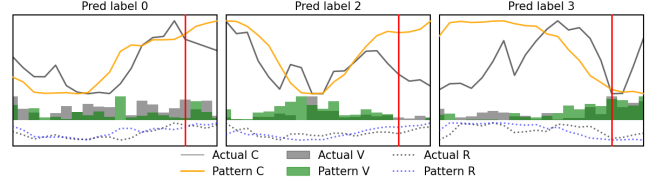


Figure 6: Failure prediction cases.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00456247, No. RS-2023-00218913) and by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2025-25443718, Next-HCAI Project) and (No. RS-2019-II191906, Artificial Intelligence Graduate School Program (POSTECH)).

A Traditional Stock Chart Patterns

To extract traditional patterns, we utilize historical daily price data from 418 companies that were constituents of the S&P 500 index at any time between January 2008 and July 2021. We base our pattern definitions on the ten canonical forms introduced by Lo et al. [18], which include: Head-and-Shoulders (HS), Inverse Head-and-Shoulders (IHS), Broadening Tops (BTOP), Broadening Bottoms (BBOT), Triangle Tops (TTOP), Triangle Bottoms (TBOT), Rectangle Tops (RTOP), Rectangle Bottoms (RBOT), Double Tops (DTOP), and Double Bottoms (DBOT). These canonical forms are **rule-based** and defined solely using price movements. To smooth the price series and reduce short-term volatility, we apply kernel regression using a Gaussian kernel with bandwidth $h = 0.3$. For peak and trough detection, we first identify local maxima and minima from the smoothed series using a broader kernel ($h = 0.8$) to account for noise. Around each detected extremum, we refine its position by searching for the maximum or minimum value within a ± 2 day range of the smoothed signal. Each candidate pattern is then evaluated based on the structural constraints defined in [18]. For example, the Head-and-Shoulders (HS) pattern consists of five consecutive extremal points where the middle peak is the highest, the two shoulders are lower and roughly symmetrical, and the outer troughs and inner troughs differ by no more than 3% of the average of the outer peaks. To capture a wide range of temporal resolutions suitable for the SIMPC algorithm, we consider all window sizes between 15 and 35 trading days. Based on this setting, we identify the most frequently occurring patterns with more than 1,000 instances in the S&P 500 dataset as follows: IHS (2,567), HS (2,525), TBOT (2,237), BTOP (1,718), TTOP (1,485), and BBOT (1,179). As shown in Fig. 7, these six patterns are used as the initial centroids in our SIMPC.

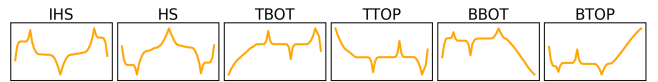


Figure 7: Six initial price patterns used as centroids in SIMPC.

GenAI Usage Disclosure

During the preparation of this paper, we used generative AI for grammar and style checking of the English abstract. No generative AI tools were used in designing experiments or writing the main content of the paper.

References

- [1] Sercan O. Arik and Tomas Pfister. 2019. TabNet: Attentive Interpretable Tabular Learning. arXiv:1908.07442 [cs.LG] Preprint.
- [2] Donald J Berndt and James Clifford. 1994. Using dynamic time warping to find patterns in time series. In *Proceedings of the 3rd international conference on knowledge discovery and data mining*. 359–370.
- [3] Aaron Bostrom and Anthony Bagnall. 2017. Binary shapelet transform for multi-class time series classification. *Transactions on Large-Scale Data- and Knowledge-Centered Systems XXXII: Special Issue on Big Data Analytics and Knowledge Discovery* (2017), 24–46.
- [4] Yuntao Du, Jindong Wang, Wenjie Feng, Sinno Pan, Tao Qin, Renjun Xu, and Chongjun Wang. 2021. AdaRNN: Adaptive Learning and Forecasting of Time Series. *CIKM* (2021).
- [5] Ambedkar Dukkipati, Kawin Mayilvahanan, Naveen Kumar Pallekonda, Sai Prakash Hadnoor, and Ranga Shaarad Ayyagari. 2025. Predictive AI with External Knowledge Infusion for Stocks. *arXiv preprint arXiv:2504.20058* (2025).
- [6] Philippe Esling and Carlos Agon. 2012. Time-series data mining. *ACM Computing Surveys (CSUR)* 45, 1 (2012), 1–34.
- [7] Jinyong Fan and Yanyan Shen. 2024. StockMixer: a simple yet strong MLP-based architecture for stock price forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8389–8397.
- [8] Josif Grabocka, Nicolas Schilling, Martin Wistuba, and Lars Schmidt-Thieme. 2014. Learning time-series shapelets. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 392–401.
- [9] Hongbin Huang, Minghua Chen, and Xiao Qiao. 2024. Generative learning for financial time series with irregular and scale-invariant patterns. In *The Twelfth International Conference on Learning Representations*.
- [10] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*.
- [11] Hyunwook Lee, Chunggi Lee, Hongkyu Lim, and Sungahn Ko. 2022. TILDE-Q: a transformation invariant loss function for time-series forecasting. *arXiv preprint arXiv:2210.15050* (2022).
- [12] Guozhong Li, Byron Choi, Jianliang Xu, Sourav S Bhowmick, Kwok-Pan Chun, and Grace Lai-Hung Wong. 2021. Shapenet: A shapelet-neural network approach for multivariate time series classification. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 8375–8383.
- [13] Shuqi Li, Weiheng Liao, Yuhan Chen, and Rui Yan. 2023. PEN: prediction-explanation network to forecast stock price movement with better explainability. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5187–5194.
- [14] Tong Li, Zhaoyang Liu, Yanyan Shen, Xue Wang, Haokun Chen, and Sen Huang. 2024. Master: Market-guided stock transformer for stock price forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 162–170.
- [15] Junhua Liu. 2024. A Survey of Financial AI: Architectures, Advances and Open Challenges. arXiv:2411.12747 [q-fin.TR] <https://arxiv.org/abs/2411.12747>
- [16] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lijun Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. *ICLR* (2024).
- [17] Andrew W. Lo. 2004. The Adaptive Markets Hypothesis: Market Efficiency from an Evolutionary Perspective. *Journal of Portfolio Management* (2004).
- [18] Andrew W. Lo, Harry Mamaysky, and Jiang Wang. 2000. Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation. *The Journal of Finance* (2000).
- [19] Ali Mehrabian, Ehsan Hoseinzade, Mahdi Mazloum, and Xiaohong Chen. 2025. Mamba meets financial markets: A graph-mamba approach for stock price prediction. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
- [20] Elizbar A Nadaraya. 1964. On estimating regression. *Theory of Probability & Its Applications* 9, 1 (1964), 141–142.
- [21] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.
- [22] Yao Qin, Dongjin Song, Haifeng Cheng, Wei Cheng, Guofei Jiang, and Garrison W. Cottrell. 2017. A dual-stage attention-based recurrent neural network for time series prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*. AAAI Press, 2627–2633.
- [23] Barbara Rossi. 2013. Exchange rate predictability. *Journal of economic literature* 51, 4 (2013), 1063–1119.
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. *NeurIPS* (2017).
- [25] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *International Conference on Learning Representations*.
- [26] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [27] Xueqing Wu, Lewen Wang, Yingce Xia, Weiqing Liu, Lijun Wu, Shufang Xie, Tao Qin, and Tie-Yan Liu. 2021. Temporally correlated task scheduling for sequence learning. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*.
- [28] Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. 2020. Qlib: An ai-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189* (2020).
- [29] Lexiang Ye and Eamonn Keogh. 2009. Time series shapelets: a new primitive for data mining. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. 947–956.
- [30] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.
- [31] Liang Zeng, Lei Wang, Hui Niu, Ruchen Zhang, Ling Wang, and Jian Li. 2024. Trade When Opportunity Comes: Price Movement Forecasting via Locality-Aware Attention and Iterative Refinement Labeling. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. 6134–6142. doi:10.24963/ijcai.2024/678 Main Track.
- [32] Chuheng Zhang, Yuanqi Li, Xi Chen, Yifei Jin, Pingzhong Tang, and Jian Li. 2020. Doubleensemble: A new ensemble method based on sample reweighting and feature selection for financial data analysis. In *Proceedings of the 2020 IEEE International Conference on Data Mining (ICDM)*.
- [33] Liheng Zhang, Charu Aggarwal, and Guo-Jun Qi. 2017. Stock price prediction via discovering multi-frequency trading patterns. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2141–2149.
- [34] Ying Zhong, Dong Huang, and Chang-Dong Wang. 2023. Deep temporal contrastive clustering. *Neural Processing Letters* 55, 6 (2023), 7869–7885.
- [35] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wencho Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *AAAI* (2021).
- [36] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning*. PMLR, 27268–27286.